Vibe-coding with AI

Everything you need to bring, install, and set up before Week 1.

What you need

Bring these to the course. Everything listed is free.

A laptop	Windows 10 or 11, or a Mac (macOS). You need to be able to install software on it. **Chromebooks and iPads will not work for this course.**
A GitHub account	Free. You will create one in Step 1 below.
GitHub Copilot access	Free for students through GitHub Education. Set up in Step 2.
Two free tools	The GitHub Copilot CLI and Git. Installed in Steps 3 and 5.
No experience	You do not need any coding or terminal experience. We teach the basics you need.

> ⚠️ **Start early**
>
> Student verification for Copilot can take 1 to 2 days. Do Steps 1 and 2 as soon as you can, not the night before class.

Setup steps

Do these before your first class session, in order.

Step 1: Create a GitHub account

1. Go to github.com/signup (https://github.com/signup)
2. Sign up with your email (a personal email is fine)
3. Choose a username you're happy with, since it'll become part of your website URL (`yourusername.github.io`)
4. Verify your email

Already have a GitHub account? Skip to Step 2.

Step 2: Get GitHub Copilot free access

GitHub Copilot is free for students through GitHub Education.

1. Go to education.github.com/benefits (https://education.github.com/benefits)
2. Click "Get student benefits"
3. You'll need to verify you're a student (school email, student ID photo, or enrollment letter)
4. Approval usually takes a few minutes but can take up to a few days

Step 3: Install the GitHub Copilot CLI

This is the tool you'll use to build everything in this course. Pick the command for your computer.

Windows:

1. Open Terminal (search "Terminal" in the Start menu)
2. Run: `winget install GitHub.Copilot`
3. Close and reopen Terminal when it finishes

Mac:

1. Open Terminal (press Cmd+Space, type "Terminal", press Enter)
2. Run: `curl -fsSL https://gh.io/copilot-install | bash`

3. Close and reopen Terminal when it finishes

> **Where's my terminal?**
>
> **Windows:**
>
> Search "Terminal" in the Start menu.
>
> **Mac:**
>
> Press Cmd+Space and type "Terminal".
>
> *On older Windows 10 machines you may not see "Terminal." Search "PowerShell" instead, or install "Windows Terminal" free from the Microsoft Store. Any of these works the same for this course.*

Step 4: Launch and sign in

1. Start the Copilot CLI by typing: `copilot`
2. On first launch, type `/login` and press Enter
3. Follow the prompts to sign in with your GitHub account (it opens a browser)
4. Once signed in, you're ready to go!

Step 5: Install Git

Git tracks your code and is what the Copilot CLI uses behind the scenes to deploy your apps.

1. Go to git-scm.com/downloads (https://git-scm.com/downloads)
2. Download and install for your system, keeping the default options
3. **Mac:** You may already have it. Run `git --version` in Terminal to check
4. Verify: run `git --version` and you should see a version number

Verify everything works

Open your terminal and run `copilot`. Once it starts, type a simple request:

```
Say hello and tell me a programming joke
```

If Copilot responds, you're all set!

Checklist:

☐ GitHub account created and email verified

☐ GitHub Education benefits applied

☐ Typing `copilot` launches the tool

☐ You signed in with `/login` and got a response to a test message

☐ `git --version` shows a version number

Troubleshooting

- **"copilot is not recognized"**: Close and reopen your terminal after installing. On Windows, you may need to restart your computer.
- **"winget is not recognized" (Windows)**: Update to the latest Windows, or install "App Installer" from the Microsoft Store, then try again.
- **"Copilot access denied"**: Make sure you ran `/login` and your Copilot access is active (check at github.com/settings/copilot).
- **Education verification pending**: You can still use the free Copilot tier while you wait. Everything in this course works with it.
- **Stuck?**: Bring your laptop to class 10 minutes early. We'll get you sorted.

Terminal Basics

Everything you need to know about the terminal for this whole course fits on this page. You will not become a terminal expert, and you do not need to. You only need a few commands.

What is the terminal?

The **terminal** is a window where you type instructions to your computer as text, instead of clicking buttons. Each instruction you type is called a **command**. You type a command, press `Enter`, and the computer does it.

That is the whole idea. Type words, press Enter, something happens. If nothing looks broken, it worked.

> **The golden rule**
>
> Type the command exactly as shown, then press `Enter`. Spelling and spaces matter. The terminal is picky, and that is normal.

Folders have another name: directories

On your computer, files live inside **folders**. In the terminal, a folder is called a **directory**. Same thing, different word. When you see "directory," think "folder."

At any moment, the terminal is "standing inside" one folder. That is your **current folder**. Commands act on wherever you currently are, so knowing where you are matters.

The only commands you need

`cd`	**Change directory.** Move into a folder. `cd ~/Documents/my-game` moves you into the `my-game` folder.
`mkdir`	**Make directory.** Create a new folder. `mkdir ~/Documents/my-game` creates a folder named `my-game`.
`ls`	**List.** Show what is inside your current folder. Type `ls` and press Enter to see the files and folders around you.
`copilot`	**Start Copilot.** Launch your AI building partner in the current folder. Always run `cd` into your project folder first, then type `copilot`.

What is the `~` symbol?

The squiggle `~` (called a tilde) is a shortcut that means **your home folder**: the folder with your name on it where your Documents, Downloads, and Desktop live. It works the same on Windows and Mac.

So `~/Documents/my-game` just means "the my-game folder, inside Documents, inside your home folder." You do not have to type your full username every time.

Moving around with `cd`

Folders are nested inside other folders, like boxes inside boxes. `cd` is how you move between them. You can move down into a folder, or back up out of one.

`cd my-game`	**Go down** into the `my-game` folder that is inside where you are now.
`cd ..`	**Go up** one level, out to the folder that contains you. The two dots mean "the folder above me."
`cd ../..`	**Go up two** levels at once. Add another `/..` for each level you want to climb.
`cd ~`	**Go home.** Jump straight back to your home folder from anywhere. A safe reset button.

Lost? Run `ls` to see what is around you, or `cd ~` to start over from home. You can always find your way back.

The pattern you will use every week

Every project in this course starts the same three-step way:

```
mkdir ~/Documents/my-game
cd ~/Documents/my-game
copilot
```

In plain English, that is:

1. **Make** a new folder for the project (`mkdir`).
2. **Move** into that folder (`cd`).
3. **Start** Copilot there (`copilot`).

After the first week, when you come back to a project you already made, you skip step 1. You only `cd` into it and start `copilot` .

> **Common mix-ups**
>
> - **"The system cannot find the path"**
>
> : the folder name is misspelled, or you have not created it yet with `mkdir` .
> - **Nothing happens after typing**
>
> : you probably forgot to press `Enter` .
> - **"copilot is not recognized"**
>
> : close and reopen the terminal, or revisit the setup steps.

> **You cannot break your computer by typing these**
>
> `cd` , `mkdir` , and `ls` are safe. The worst that happens is an error message, which just means "try again." Copilot will never run a command without showing you first and asking.

Prompting Copilot Cheat Sheet

Quick reference for writing effective prompts. Keep it handy while you work.

The Prompt Formula

```
[Action] + [What] + [Details] + [Format/Constraints]
```

Part	Purpose	Example
Action	What Copilot should do	Create, Fix, Add, Explain, Refactor
What	The thing you're building/changing	a tic-tac-toe game, a timer function
Details	Specific behavior and features	that tracks score, with a reset button
Format	Technical constraints	in a single HTML file, using CSS grid

Good Prompts vs Bad Prompts

❌ **Bad**

"make a game"

"fix it"

"make it look better"

✓ **Good**

"Create a memory-match card game in a single HTML file with a 4x4 grid"

"The timer doesn't stop when it reaches zero, so add a check that stops the interval"

"Change the background to a dark theme with white text and blue accents"

Useful Prompt Starters

- **Building:** "Create a [thing] that [behavior] in a single HTML file"

- **Adding features:** "Add [feature] to my existing code. It should [behavior]"
- **Fixing bugs:** "I expected [X] but got [Y]. Here's the code: [paste]. Fix it."
- **Understanding:** "Explain what this code does line by line: [paste]"
- **Styling:** "Restyle this to look [modern/minimal/colorful/dark] with [specific details]"
- **Iteration:** "This works, but now I want to also [new requirement]"

The Debugging Formula

```
1. What I expected: [describe expected behavior]
2. What actually happens: [describe actual behavior]
3. Here's the relevant code: [paste it]
4. How do I fix this?
```

When Copilot Gets It Wrong

1. **Be more specific**: add details you left out
2. **Show it the error**: paste error messages directly
3. **Ask for a simpler version**: "Start with just [core feature], nothing else"
4. **Try a different approach**: "Instead of [X], try using [Y]"
5. **Start fresh**: "Let's start over. New approach: [describe]"

AI & Copilot Glossary

Key terms you'll encounter in this course.

Core AI Concepts

LLM (Large Language Model) *(Week 1)*
A type of AI that predicts text. Given some input text, it predicts what should come next. Trained on massive amounts of human-written text and code. Examples: GPT-4, Claude, the models behind Copilot.

Model *(Week 1)*
A specific trained version of an AI system. Different models have different sizes, speeds, and specializations. Copilot uses code-specialized models.

Prompt *(Week 2)*
The text you give to an LLM to steer its predictions. Better prompts (more specific, more context) produce better output.

Context / Context Window *(Week 2)*
Everything the LLM can "see" when making predictions: your prompt, open files, chat history. Has a size limit; the model forgets anything outside the window.

Hallucination *(Week 3)*
When an LLM generates confident-sounding output that is factually wrong. Happens because it predicts likely text, not verified truth. Always test AI output.

Token
The basic unit of text an LLM processes, roughly 3/4 of a word. Context windows are measured in tokens (e.g., 128K tokens is about a short novel).

Copilot Ecosystem

Agent *(Week 4)*
An LLM that can take actions (edit files, run commands) in an autonomous loop. Unlike chat, an agent acts and checks its own work rather than just giving you text to copy.

MCP (Model Context Protocol) *(Week 5)*
A standard protocol for connecting AI to external tools and data sources. Like Bluetooth for AI: one universal way to plug in any tool (weather APIs, databases, calendars, etc.).

Skill *(Week 6)*

A packaged set of instructions that gives an AI agent a specific reusable capability. Like an app on your phone: extends what the AI can do without rebuilding from scratch.

Hook *(Week 7)*

An automated action triggered by a specific event. "When X happens, automatically do Y." Examples: auto-format on save, auto-test on commit, auto-deploy on push.

Web Development

HTML (HyperText Markup Language)

The structure/skeleton of a web page. Defines what elements exist: headings, paragraphs, buttons, images, links.

CSS (Cascading Style Sheets)

The visual styling of a web page. Defines how elements look: colors, sizes, fonts, layout, animations.

JavaScript (JS)

The programming language that makes web pages interactive. Handles logic: what happens when you click, calculations, data storage, API calls.

API (Application Programming Interface)

A way for programs to talk to each other over the internet. Your weather app calls a weather API to get data: it sends a request and gets structured data back.

fetch()

A JavaScript function that calls an API. Sends a request to a URL and returns the response. Used whenever your app needs external data.

localStorage

A browser feature that saves data on the user's computer. Persists between page loads: your timer remembers sessions, your tracker remembers expenses.

DOM (Document Object Model)

The browser's internal representation of your HTML page. JavaScript manipulates the DOM to update what users see without reloading the page.

Event / Event Listener

Something that happens (a click, a keypress, a timer firing) and code that runs in response. `addEventListener('click', fn)` = "when clicked, run this function."

Tools & Workflow

GitHub Copilot CLI

The AI coding assistant you use in this course. Launched by typing `copilot` in your terminal. It chats with you, creates and edits files, and can run commands for you (like deploying to GitHub

Pages). Installed via WinGet (Windows) or an install script (Mac).

Terminal

The text-based window where you type commands. Called Terminal on both Windows and Mac. Where you launch the Copilot CLI.

Command

A single text instruction you type into the terminal and run by pressing Enter. In this course you only need three: `mkdir`, `cd`, and `copilot`.

Directory

Another word for a folder. `mkdir` makes one, `cd` moves you into one. See the Terminal Basics section.

Git

Version control software that tracks changes to your code over time. Like "save points" in a video game: you can always go back.

GitHub

A website that stores your git repositories online. Also where Copilot, GitHub Pages, and collaboration features live.

GitHub Pages

Free web hosting from GitHub. Push your HTML/CSS/JS to a repo and get a live URL. Your apps live here.

Repository (Repo)

A project folder tracked by git. Contains your code, its history, and lives on GitHub.

Commit

A saved snapshot of your code at a point in time. Like pressing "save" but with a note about what changed.

Push

Upload your local commits to GitHub. After pushing, your live GitHub Pages site updates.